

מסמך אפיון בדיקות

כלים ואסטרטגיה

ארבע שכבות, תואמות לרשימת הכלים המקובלים של המטלה עצמה (7§: Vitest, Jest, React Testing Library, Playwright, או בדיקות ידניות מתועדות):

שכבה	כלי	היכן	למה משמשת
יחידה (Unit)	Vitest, סביבת node	**/__tests__/*.test.ts	לוגיקה טהורה: אימות Zod, חישובי תוקף, ההיוריסטיקה של השיבוץ, שומרי הרשאות
רכיב (Component)	Vitest + React Testing Library, jsdom	**/__tests__/*.test.tsx	התנהגות רכיבי לקוח (ממשק אופטימי, רינדור מותנה) ללא דפדפן אמיתי
אינטגרציה	Vitest, פרויקט Supabase אמיתי	tests/integration/*.test.ts	Server Actions אמיתיים רצים מול מדיניות RLS אמיתית ומסד Postgres אמיתי
ידני	רשימת בדיקה מתועדת	קובץ זה, "בדיקות ידניות מתועדות" בהמשך	דברים שרק דפדפן מרונדר אמיתי יכול לתפוס (ראו בהמשך)

איך שכבת האינטגרציה עובדת: היא קוראת ל-Server Actions האמיתיים, הבלתי-משונים (createShift, submitAvailability, generateSchedule, publishShift וכו') – לא מימוש מחדש של הלוגיקה שלהם. שני חלקים ספציפיים ל-Next.js בלבד שקיימים רק בתוך בקשת HTTP אמיתית מוחלפים בכפיל בדיקה: createClient() של @/lib/supabase/server (מדומה כך שמחזירה לקוח supabase-js אמיתי, כבר מחובר, עבור חשבון בדיקה זמני של מנהל/עובד שנוצר דרך ה-API Auth Admin), ו-revalidatePath של next/cache (no-op – היא זקוקה ל-store של static-generation שלא קיים מחוץ לבקשה). כל השאר – הקוד של ה-action עצמו, Postgres, וכל מדיניות RLS – אמיתי. חשבונות בדיקה נוצרים טריים ונמחקים ב-afterAll; כל שורה שבדיקה יוצרת מסומנת בקידומת __test__ (ראו tests/integration/helpers.ts) כך שניקוי שנכשל תמיד קל לזהות ולעולם לא מתבלבל עם נתונים אמיתיים/זרועים.

1. בדיקות לפיצ'רים המרכזיים

פיצ'ר	בדיקה	קובץ
חישוב תוקף/חידוש הסמכה	computeExpiresOn – לעולם לא פג, מרווח פשוט, הארכה על ידי תאריך חידוש, שוויון על הגבול, מעבר חודש/שנה	features/worker-qualifications/__tests__/computeExpiresOn.test.ts

פיצ'ר	בדיקה	קובץ
ההיריסטיקה של השיבוץ	התאמת הסמכות, התאמת זמינות, סדר לפי נדירות, שובר שוויון הוגנות, כל 3 סוגי הזיווג	features/scheduling/___tests/heuristic.test.ts (מקרים, נבנה ב-Phase 5)
הגשת זמינות (עובד)	toggle אופטימי, מצב כפתור נכון, קריאה ל-submitAvailability עם מזהי המשמרת הנכונים (כולל משמרות מקובצות/חופפות)	features/availability/___tests/AvailabilityShiftRow.test.tsx
שומרי הרשאה	requireAdmin / requireWorker מחזירות את הפרופיל הנכון לתפקיד הנכון	lib/___tests/auth.test.ts
תהליך ליבה מקצה-לקצה	יצירת משמרת → הגשת זמינות → יצירת שיבוץ → פרסום → העובד רואה אותו	tests/integration/core-flow.test.ts (מספק גם את §3 בהמשך)

2. בדיקות לקליטים לא תקינים

באפליקציה יש כ-10 סכמות Zod (אחת לכל Server Action), אך הן חוזרות על מספר קטן של צורות אימות בסיסיות: שדה חובה, בדיקת פורמט, וולידציה שתלויה בשני שדות יחד. הטבלה מכסה דוגמה אחת מייצגת לכל צורה, לא את כל 10 הסכמות — בהתאם להיתר של המטלה שלא כל שורת קוד חייבת בדיקה משלה:

סכמה	מקרים מכוסים	קובץ
shiftSchema	תאריך חסר, שעת סיום שווה ל/לפני שעת התחלה (refine . חוצה-שדות)	features/shifts/___tests/validation.test.ts
createWorkerSchema	שם ריק, אימייל שגוי, סיסמה מתחת ל-8 תווים	features/accounts/___tests/validation.test.ts
qualificationSchema	שם ריק, תקופת חידוש שלילית/לא שלמה	features/qualifications/___tests/validation.test.ts
windowSchema	תווית ריקה, closesAt שווה ל/לפני (refine . opensAt חוצה-שדות)	features/availability-windows/___tests/validation.test.ts

3. בדיקות לתהליכים עסקיים מרכזיים

מחזור החיים המלא של לוח משמרות מוצע, רץ מול פרויקט Supabase אמיתי: tests/integration/core-flow.test.ts —

- המנהל יוצר תפקיד, חלון זמינות, ומשמרת עם משבצת פתוחה אחת.
- העובד מסמן את עצמו זמין.

3. המנהל מייצר את לוח המשמרות — נבדק שהמשבצת מתמלאת על ידי העובד הזמין (קריאה אמיתית ל-`runSchedulingHeuristic`, כתיבה אמיתית ל-DB).
4. העובד עדיין לא רואה את השיבוץ (כלל תזמון הפרסום — ראו §4).
5. המנהל מפרסם את המשמרת.
6. העובד רואה עכשיו גם את השיבוץ וגם התראה, נקרא דרך ה-session האמיתי של העובד עצמו (לא לקוח service-role) — מוכיח ש-RLS באמת מעניק את הגישה, לא רק שהשורה קיימת.

4. בדיקות הרשאות (תפקידי משתמש שונים)

מה	שכבה	קובץ
<code>requireAdmin / requireWorker</code> מפנות תפקיד שגוי, מחזירות את הפרופיל לתפקיד הנכון	שומר אפליקציה (DB מדומה, מהיר)	<code>lib/__tests__/auth.test.ts</code>
עובד שקורא ל-Server Action שמורה-למנהל מופנה (לא רק מסורב נתונים)	שומר אפליקציה, session אמיתי של Supabase	<code>tests/integration/authorization.test.ts</code>
מנהל שקורא לאותה פעולה מצליח מקצה לקצה	שומר אפליקציה + RLS, Supabase אמיתי	אותו קובץ
עובד לא יכול לקרוא שורות <code>worker_qualifications</code> של עובד אחר	RLS	אותו קובץ
עובד לא יכול ליצור או אפילו לקרוא <code>shift_templates</code> (מנהל בלבד, אפס נראות לעובד)	RLS	אותו קובץ
עובד לא רואה את השיבוץ שלו לפני שהמשמרת פורסמה, רק אחרי; המנהל רואה בכל מקרה	RLS (מבוטא-תזמון, לא רק תפקיד)	אותו קובץ

5. בדיקות למסד הנתונים

- התנהגות אמיתית של מפתח זר/אילוץ ייחודיות מופעלת באופן משתמע על ידי כל בדיקת אינטגרציה (למשל `positions.name unique`, מחיקות מדורגות (cascade) על משתמשי בדיקה זמניים המנקות אוטומטית `(worker_qualifications / availability / assignments)`).
- טריגרים ברמת ה-DB (אפשרות-נדרשת-אם-ורק-אם-להסמכה-יש-אפשרויות וכו') אומתו התנהגותית מול הכנסות אמיתיות במהלך Phase 3 (ראו `TODO.md`) ולא נבדקו שוב כאן — אלה אילוצים יציבים, בעלי שינוי נמוך, שלא השתנו מאז.

6. בדיקות למקרי קצה

מקרה קצה	קובץ
זיווג <code>avoid</code> קשיח משאיר משבצת ריקה במקום לזווג שני עובדים	<code>features/scheduling/__tests__/heuristic.test.ts</code>
<code>prefer_avoid</code> רק מזווג רק כשזו האפשרות היחידה, ומסומן	אותו קובץ
<code>min_rest_hours</code> משנה משהו רק בין תאריכים שונים (אותו תאריך חסום מבנית על ידי מניעת שיבוץ כפול קודם)	אותו קובץ
חריגת אילוף לפי קטגוריה שולטת במלואה (כולל דגל ה- <code>enabled</code> שלה) כשהיא תואמת, ללא תלות במצב שורת ברירת המחדל	אותו קובץ
תאריך חידוש הסמכה שווה בדיוק ל- <code>obtainedAt</code> (גבול ההשוואה <code>></code>)	<code>features/worker-qualifications/__tests__/computeExpiresOn.test.ts</code>
גבולות ולידציה חוצת-שדות (שעת סיום שווה לשעת התחלה; <code>closesAt</code> שווה ל- <code>opensAt</code>)	<code>features/shifts/__tests__/validation.test.ts</code> , <code>features/availability-windows/__tests__/validation.test.ts</code>
העובד רואה אפס שורות (לא שגיאה) כש-RLS מסכן אותן – מקרה הקצה של "אין נתונים" עבור גבול אבטחה, לא רק מצב ריק רגיל	<code>tests/integration/authorization.test.ts</code>

7. בדיקות ממשק בסיסיות

- `features/availability/__tests__/AvailabilityShiftRow.test.tsx` – רינדור מותנה של `badge, variant` של הכפתור המשקף את המצב, ושכל ערך תאריך/שעה מרונדר בתוך אלמנט עם `dir="ltr"` (כדי למנוע היפוך חזותי של המספרים תחת RTL).
- כל מה שבאמת חזותי – פריסת RTL בפועל, היפוך הסדר החזותי של תאריכים/שעות תחת `dir="rtl"` ללא `dir="ltr"` מפורש (למשל 12:30 מוצג כ- 30:12), נקודות שבירה רספונסיביות – לא ניתן לתפוס על ידי `jsdom` (הוא לא מפרס שום דבר ולא מפעיל את אלגוריתם ה-bidi של יוניקוד) ואין נתיב אוטומציית-דפדפן מקומי שעובד. אלה מכוסים על ידי רשימת הבדיקה הידנית בהמשך במקום זאת.

בדיקות ידניות מתועדות

לפי רשימת הכלים המקובלים של המטלה עצמה (§7). לבדוק מחדש ידנית את הרשימה הזו אחרי כל שינוי שנוגע לתחומים הבאים – כל פריט כבר אומת לפחות פעם אחת במהלך הפיתוח, כך שזו רשימת רגרסיה, לא חקירה ראשונית:

- כל קלט ותצוגת תאריך/שעה מרונדרים משמאל-לימין וקריאים תחת `dir="rtl"` (יצירה/עריכה ב- `/admin/shifts`, רשימת המשמרות, `/admin/availability-windows`, תאריכי השגה/פקיעה ב- `/availability`, `/my-qualifications`).

- כותרת האפליקציה, תפריט ה-hamburger, ואריחי הניווט מרונדרים ונפתחים נכון גם במסך קטן (מובייל, כ-375px) וגם גדול (דסקטופ, כ-1440px).
- עמודי CRUD של המנהל (הסמכות/תפקידים/משמרות/תבניות משמרת/עובדים) שמישים ברוחב מובייל ללא גלישה אופקית או פקדים חתוכים.
- חשבון עובד שזה עתה נוצר יכול באמת להתחבר עם הסיסמה שהמנהל הגדיר, ונוחת על דשבורד העובד, לא דשבורד המנהל.
- באנר הקונפליקט `prefer_avoid` במסך בחינת לוח המשמרות גלוי גם לפני פרסום וגם אחריו (דגל מחושב-חי, לא תמונת מצב חד-פעמית).
- כלל תזמון הפרסום של RLS מתקיים בממשק בפועל, לא רק בשאלות ישירות: עמוד `/my-shifts` של עובד לא מציג כלום עבור משמרת לא-מפורסמת ומציג את השיבוץ האמיתי ברגע שהיא פורסמת.
- תצוגת היומן של `/admin/shifts` : מתגי רשימה/יומן וחודש/שבוע מתחלפים נכון; לחיצה על משמרת בטייטה/משובצת פותחת את טופס העריכה שלה בכרטיס השמאלי ולחיצה על משמרת מפורסמת פותחת במקום זאת את התצוגה לקריאה-בלבד; שמירה/ביטול/מחיקה מטופס העריכה כולם חוזרים נכון ל"הוספת משמרת חדשה"; ה-tooltip שמופיע ב-hover מופיע גם על ציפים בתצוגת חודש וגם על כרטיסים בתצוגת שבוע ונעלם עם יציאת העכבר.